

LOGIC BUILDING MASTERY GUIDE

A Step-by-Step System to Solve Any Problem

From Zero to Logical Thinking — with Real Code Examples



Think Like a Programmer



Pattern Recognition



Step-by-Step Framework



10 Solved Problems

SECTION 1 | What is Logic Building?

What is Logic Building?

Logic building is the ability to break a complex problem into small, clear, manageable steps and then translate those steps into code or a solution a computer can execute. It is the single most important skill for any programmer — more important than knowing any specific language.

Why is it Hard for Beginners?

- They try to write code before understanding the problem fully.
- They think in human language but code requires machine-precise logic.
- They skip the planning phase and jump straight to syntax.
- They don't break problems into sub-problems — they see a wall, not bricks.

The Good News

Logic building is a **learnable skill**. Like a muscle, it grows stronger every time you practice. This guide gives you a repeatable framework — a set of steps and patterns — you can apply to ANY problem, forever.

The 3 Pillars of Logical Thinking

PILLAR	MEANING	QUESTION TO ASK
■ Understand	Grasp what the problem truly needs	What is the input? What is the output?
>■ Decompose	Break big into small	What are the sub-tasks?
■ Pattern	Reuse known solutions	Have I seen something like this before?

SECTION 2 | The 7-Step Problem-Solving Framework

Apply These 7 Steps to Every Problem

Whether you are solving a LeetCode problem, building a feature, or debugging — these 7 steps will guide you from confusion to clarity every single time.

1 READ & UNDERSTAND

Read the problem at least twice. Don't rush. Identify: What are the inputs? What should the output be? Are there constraints (size, time, negatives)?

2 WORK EXAMPLES BY HAND

Before writing code, solve 2-3 small examples manually on paper or mentally. Trace the logic with real numbers. This reveals the pattern.

3 IDENTIFY THE PATTERN

Ask: Is this a loop? A condition? A search? Sorting? Counting? Map it to a known category. Most problems are variations of 10-15 core patterns.

4 WRITE PSEUDOCODE

Write plain English steps (not code). e.g. 'For each number in the list, if it is even, add it to the sum.' This bridges thinking and coding.

5 TRANSLATE TO CODE

Now convert your pseudocode to actual code line by line. Don't worry about perfection. Get it working first — optimize later.

6 TEST WITH EDGE CASES

Test with: empty input, single element, very large input, negative numbers, duplicate values. Edge cases reveal hidden bugs.

7 REFLECT & OPTIMIZE

Ask: Is this the simplest solution? Can I reduce time or space complexity? What did I learn? Write it down to reinforce the pattern.

SECTION 3 | 10 Core Logic Patterns You Must Know

90% of all programming problems reduce to one of these 10 patterns. Recognizing the pattern is the biggest step toward the solution.

#	PATTERN	WHEN TO USE	KEY CLUE
1	Counting / Accumulator	Sum, count, product of items	"find total", "how many"
2	Linear Search	Find an item in a list	"find", "exists", "search"
3	Two Pointers	Pairs, sorted arrays, palindromes	"two elements", "pair sum"
4	Sliding Window	Subarrays, substrings of fixed size	"consecutive", "window"
5	Frequency Map	Count occurrences, duplicates	"most frequent", "duplicate"
6	Sorting First	Easier after ordering	"smallest k", "median", "range"
7	Recursion / Divide	Problem = smaller version of itself	"tree", "factorial", "subsets"
8	Greedy	Best local choice = best global	"minimum steps", "maximum profit"
9	Binary Search	Fast lookup in sorted data	"sorted", "find target", " $O(\log n)$ "
10	Stack / Queue	Track order, nesting, history	"brackets", "next greater", "BFS"

SECTION 4 | 10 Solved Problems with Full Walkthrough

Each problem below is solved using the 7-Step Framework. Follow every step — especially the hand-trace and pseudocode — before reading the code.

Problem 1 | Sum of All Even Numbers in a List

Pattern: Counting / Accumulator

Input: `numbers = [1, 2, 3, 4, 5, 6]`

Output: 12 (2+4+6)

Step 2 — Work by Hand

Go through each number. If it divides evenly by 2, add it to a running total.

Step 4 — Pseudocode

Pseudocode

```
SET total = 0
FOR each number in list:
    IF number % 2 == 0:
        ADD number to total
RETURN total
```

Step 5 — Python Code

Python

```
def sum_even(numbers):
    total = 0
    for num in numbers:
        if num % 2 == 0: # even check
            total += num
    return total

# Test
print(sum_even([1,2,3,4,5,6])) # 12
```

Problem 2 | Check if a String is a Palindrome

Pattern: Two Pointers

Input: `"racecar"`

Output: `True`

Step 2 — Work by Hand

Compare first and last characters: `r==r`, `a==a`, `c==c` — all match. It is a palindrome.

Step 4 — Pseudocode

Pseudocode

```
SET left = 0, right = len(s) - 1
WHILE left < right:
    IF s[left] != s[right]: RETURN False
    INCREMENT left, DECREMENT right
RETURN True
```

Step 5 — Python Code

Python

```
def is_palindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True

print(is_palindrome('racecar')) # True
print(is_palindrome('hello'))  # False
```

Problem 3 | Find the Largest Number

Pattern: Linear Search / Accumulator

Input: [3, 7, 1, 9, 4]

Output: 9

Step 2 — Work by Hand

Start with first element as max. Compare each: 7>3, keep 7. 9>7, keep 9. Done.

Step 4 — Pseudocode

Pseudocode

```
SET max_val = first element
FOR each number in list:
    IF number > max_val:
        SET max_val = number
RETURN max_val
```

Step 5 — Python Code

Python

```
def find_max(numbers):
    max_val = numbers[0]
    for num in numbers:
        if num > max_val:
            max_val = num
    return max_val

print(find_max([3,7,1,9,4])) # 9
```

Problem 4 | Count Frequency of Each Character

Pattern: Frequency Map (Dictionary)

Input: "hello"

Output: {'h':1, 'e':1, 'l':2, 'o':1}

Step 2 — Work by Hand

Walk through each letter. Keep a dictionary. h->1, e->1, l->1, l->2, o->1.

Step 4 — Pseudocode

Pseudocode

```
SET freq = empty dictionary
FOR each char in string:
    IF char in freq: freq[char] += 1
    ELSE: freq[char] = 1
RETURN freq
```

Step 5 — Python Code

Python

```
def char_frequency(s):
    freq = {}
    for char in s:
        if char in freq:
            freq[char] += 1
        else:
            freq[char] = 1
    return freq

print(char_frequency("hello"))
# {'h':1, 'e':1, 'l':2, 'o':1}
```

Problem 5 | Fibonacci Sequence (First N Terms)

Pattern: Accumulator + Pattern Recognition

Input: `n = 7`

Output: `[0, 1, 1, 2, 3, 5, 8]`

Step 2 — Work by Hand

Start: 0, 1. Each next = sum of previous two: $0+1=1$, $1+1=2$, $1+2=3$, $2+3=5$, $3+5=8$.

Step 4 — Pseudocode

Pseudocode

```
IF n == 1: RETURN [0]
IF n == 2: RETURN [0, 1]
SET seq = [0, 1]
REPEAT (n-2) times:
    APPEND seq[-1] + seq[-2] to seq
RETURN seq
```

Step 5 — Python Code

Python

```
def fibonacci(n):
    if n == 1: return [0]
    seq = [0, 1]
    for i in range(2, n):
        seq.append(seq[-1] + seq[-2])
    return seq

print(fibonacci(7)) # [0,1,1,2,3,5,8]
```

Problem 6 | Find Two Numbers That Sum to Target

Pattern: Frequency Map (Hash Set)

Input: `nums=[2,7,11,15], target=9`

Output: `[0, 1]` (indices of 2 and 7)

Step 2 — Work by Hand

For each number, check if $(\text{target} - \text{number})$ was seen before. $9-2=7$; see 7? Yes!

Step 4 — Pseudocode

Pseudocode

```
SET seen = empty map {value -> index}
FOR i, num in enumerate(nums):
    SET complement = target - num
    IF complement in seen:
        RETURN [seen[complement], i]
    seen[num] = i
```

Step 5 — Python Code

Python

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

print(two_sum([2,7,11,15], 9)) # [0, 1]
```

Problem 7 | Reverse a List Without Built-ins

Pattern: Two Pointers

Input: [1, 2, 3, 4, 5]

Output: [5, 4, 3, 2, 1]

Step 2 — Work by Hand

Swap first and last: 1<->5. Swap second and second-to-last: 2<->4. Middle stays.

Step 4 — Pseudocode

Pseudocode

```
SET left = 0, right = len(arr) - 1
WHILE left < right:
    SWAP arr[left] and arr[right]
    INCREMENT left, DECREMENT right
RETURN arr
```

Step 5 — Python Code

Python

```
def reverse_list(arr):
    left, right = 0, len(arr) - 1
    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1
    return arr

print(reverse_list([1,2,3,4,5])) # [5,4,3,2,1]
```

Problem 8 | Check Balanced Parentheses

Pattern: Stack

Input: "(())"

Output: True

Step 2 — Work by Hand

Push open brackets. When a close bracket appears, pop and check match. If stack empty at end -> balanced.

Step 4 — Pseudocode

Pseudocode

```
SET stack = []
FOR each char in string:
    IF char is opening: PUSH to stack
    IF char is closing:
        IF stack empty OR top doesn't match: RETURN False
        POP from stack
RETURN stack is empty
```

Step 5 — Python Code

Python

```
def is_balanced(s):
    stack = []
    pairs = {'(': '(', ')': ')', '[': '[', ']': ']'}
    for ch in s:
        if ch in '({[':
            stack.append(ch)
        elif ch in ')}]':
            if not stack or stack[-1] != pairs[ch]:
                return False
            stack.pop()
    return len(stack) == 0

print(is_balanced("({}{}[]")) # True
```

Problem 9 | Binary Search in Sorted List

Pattern: Binary Search (Divide and Conquer)

Input: `arr=[1,3,5,7,9,11]`, `target=7`

Output: 3 (index)

Step 2 — Work by Hand

Mid = index 2 (value 5). $7 > 5$, search right half. Mid = index 4 (value 9). $7 < 9$, search left. Mid=3 (value 7). Found!

Step 4 — Pseudocode

Pseudocode

```
SET low=0, high=len(arr)-1
WHILE low <= high:
    SET mid = (low + high) // 2
    IF arr[mid] == target: RETURN mid
    IF arr[mid] < target: low = mid + 1
    ELSE: high = mid - 1
RETURN -1
```

Step 5 — Python Code

Python

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

print(binary_search([1,3,5,7,9,11], 7)) # 3
```

Problem 10 | FizzBuzz (Classic Pattern Problem)

Pattern: Condition Chains

Input: `n = 15`

Output: 1,2,Fizz,4,Buzz,...,FizzBuzz

Step 2 — Work by Hand

For 1-15: if divisible by both 3&5 -> FizzBuzz. By 3 only -> Fizz. By 5 only -> Buzz. Else -> number.

Step 4 — Pseudocode

Pseudocode

```
FOR i from 1 to n:
    IF i % 3 == 0 AND i % 5 == 0: PRINT FizzBuzz
    ELSE IF i % 3 == 0: PRINT Fizz
    ELSE IF i % 5 == 0: PRINT Buzz
    ELSE: PRINT i
```

Step 5 — Python Code

Python

```
def fizzbuzz(n):
    result = []
    for i in range(1, n + 1):
        if i % 3 == 0 and i % 5 == 0:
            result.append('FizzBuzz')
        elif i % 3 == 0:
            result.append('Fizz')
        elif i % 5 == 0:
            result.append('Buzz')
        else:
            result.append(str(i))
    return result

print(fizzbuzz(15))
```

SECTION 5 | Your 30-Day Logic Building Plan

How to Practice Consistently

Logic builds through repetition. Here is a structured 30-day plan. Spend 30-60 minutes per day. Quality beats quantity — understand each solution deeply.

WEEK	FOCUS	DAILY ACTIVITY
Week 1 (Days 1–7)	Foundations	Loops, conditions, functions. Solve 1 accumulator + 1 search problem per day.
Week 2 (Days 8–14)	Strings & Lists	Palindromes, reversal, frequency. Solve 2 problems per day using Pseudocode first.
Week 3 (Days 15–21)	Maps & Sorting	Two Sum, duplicates, sorting patterns. Solve 2-3 problems, trace by hand before coding.
Week 4 (Days 22–30)	Advanced Patterns	Stack, Binary Search, Recursion. Solve 3 problems/day + revisit 1 old problem.

Golden Rules to Always Follow

- **Never:** Skip straight to code without thinking first.
- **Always:** Solve 2 examples by hand before writing a single line of code.
- **Always:** Write pseudocode — even 3 lines — before actual code.
- **Repeat:** After solving, ask: what pattern did this use? Write it down.
- **Track:** Keep a log of problems solved + patterns used. Review weekly.
- **Test:** Run your code on edge cases: empty, single, large, negative.

Remember This Always

Every expert programmer was once a beginner who couldn't solve FizzBuzz. The only difference between them and you is practice and patience.

You now have the Framework. You have the Patterns. You have 10 Solved Examples. All you need to do is
START — one problem at a time.
